

Simulación del Juego Duck Hunt usando Python

Maestría en Ciencia de Datos e Inteligencia Artificial — UTEC

Julio de 2026

Índice

1. Introducción	1
2. Objetivos	1
3. Descripción de la solución	2
4. Funcionalidades implementadas	5
5. Librerías utilizadas	5
6. Análisis estadístico de resultados	6
7. Decisiones de diseño	6
8. Conclusiones	7
9. Cómo ejecutar el proyecto	7

Informe técnico — Proyecto de curso, Programación 101

Curso: Programación 101 **Profesor:** Royer Rojas Malasquez **Integrantes del equipo:** *[completar con los nombres del equipo antes de entregar]* **Repositorio:** github.com/GoDeeper96/duckHuntProyectoMaestria

1. Introducción

Este informe documenta el desarrollo de una simulación del clásico videojuego **Duck Hunt**, construida en Python como proyecto de curso de Programación 101. La idea central es sencilla de contar: en un tablero dividido en una cuadrícula, aparece un pato en una celda elegida al azar, se dispara a otra celda también elegida al azar, y el programa valida si ambas coinciden. Esto se repite un número configurable de veces, y al final se presentan las estadísticas de toda la partida.

Detrás de esa idea simple está el objetivo real del curso: practicar programación estructurada y modular, manejo de matrices e imágenes, generación de números aleatorios, y estadística básica, usando cuatro librerías de ciencia de datos que son estándar en cualquier flujo de trabajo con Python: **NumPy**, **pandas**, **Matplotlib** y **Seaborn**.

Todo el proyecto vive en un único notebook, [DuckHunt_Simulacion.ipynb](#), organizado en 9 secciones que se ejecutan de arriba hacia abajo.

2. Objetivos

Objetivo general: desarrollar una aplicación en Python que simule el funcionamiento básico de Duck Hunt mediante una cuadrícula de posiciones aleatorias, aplicando las herramientas de procesamiento numérico, manejo de datos y visualización que son el eje del curso.

Objetivos específicos:

- Manipular matrices y arreglos con NumPy (conversión de imágenes, construcción del grid, generación de posiciones aleatorias).

- Gestionar datos estadísticos con pandas (registro de cada disparo, resumen de la partida, historial acumulado entre partidas).
- Crear gráficos con Matplotlib y Seaborn (tablero del juego, pantallas de reacción, gráficos de resultados).
- Implementar la lógica de una simulación computacional simple, con funciones bien separadas y responsabilidades claras.
- Integrar procesamiento de imágenes y visualización de datos en un mismo proyecto coherente.

3. Descripción de la solución

El notebook está dividido en 9 secciones, cada una con su propia celda de explicación en Markdown antes del código. La tabla resume qué hace cada una:

Sección	Qué hace
1. Configuración inicial	Importa las librerías, define las rutas a los archivos de <code>assets/</code> , valida que existan.
2. Configuración de la partida	Pide por teclado el nombre del jugador, el número de disparos y el tamaño del grid; valida los tres valores.
3. Carga y preprocesamiento del fondo	Carga la imagen de fondo y la convierte a escala de grises; construye la cuadrícula.
4. Función <code>pato()</code>	Elige una celda al azar y dibuja el sprite del pato ahí.
5. Función <code>pistola()</code>	Elige una celda de impacto al azar y dibuja una mira.
6. Validación de impacto	Compara la posición del pato con la del disparo; muestra WINNER o GAME OVER.
7. Bucle principal	Repite el ciclo completo tantas veces como disparos se configuraron.
8. Resultados y CSV	Arma la tabla de la partida, calcula el resumen, lo guarda en un historial.
9. Visualización estadística	Genera los gráficos finales y compara contra el historial de partidas.

3.1 El fondo del juego

El enunciado pide mostrar el fondo en blanco y negro o en escala de grises. El proyecto usa **únicamente escala de grises**, tanto para la vista de comparación como para el tablero donde realmente se juega. La conversión se hace con la fórmula de luminosidad, aplicada a toda la imagen de una sola vez con NumPy:

```
gris = 0.299·R + 0.587·G + 0.114·B
```

El tablero se divide en una cuadrícula `n x n` calculando los bordes de cada celda con `np.linspace`, lo que conecta la matriz lógica del juego (fila, columna) con la región de píxeles real que le corresponde en la imagen.

3.2 Las funciones `pato()` y `pistola()`

Ambas funciones eligen una celda al azar con `np.random.randint` y dibujan algo encima del tablero ya abierto, usando `ax.imshow(..., extent=[...])` para el pato y un `Rectangle` + marcador "x" para la mira de la pistola. Ninguna de las dos necesita mezclar píxeles a mano: Matplotlib compone la transparencia del sprite automáticamente, y las formas de la mira son piezas ya construidas de la librería.

Un detalle del proceso vale la pena mencionar: el sprite original del pato (`pato.png`) no tenía transparencia real, tenía un fondo celeste sólido dibujado como parte de la imagen. Se detectó inspeccionando el canal alfa con NumPy, y se resolvió preparando **una sola vez, de antemano**, una versión limpia del sprite

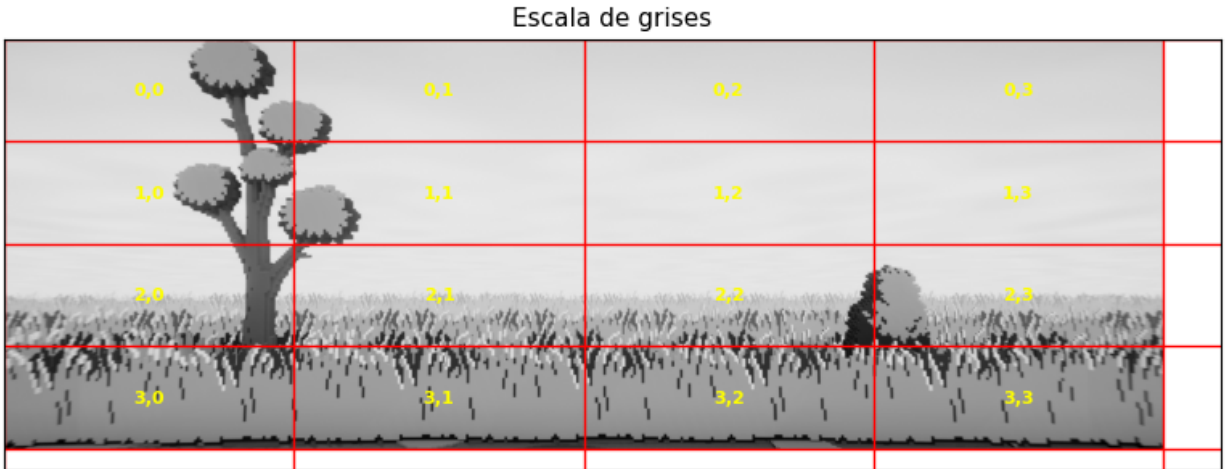


Figura 1: Fondo en escala de grises con la cuadrícula superpuesta

(`pato_limpio.png`) con ese fondo quitado — así el código de `pato()` se mantiene enfocado en su tarea principal.

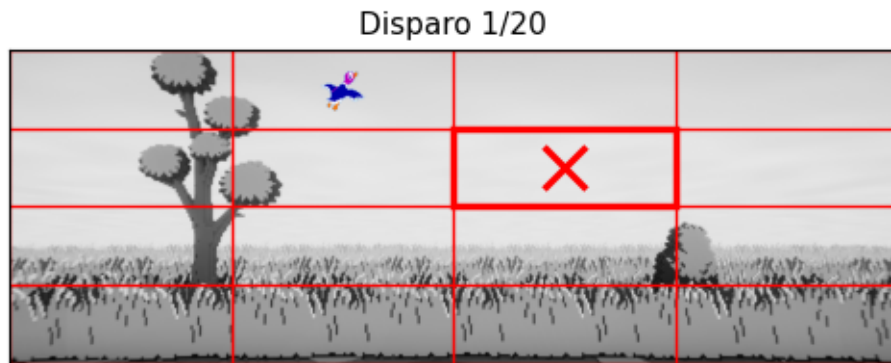


Figura 2: El pato y la mira dibujados juntos sobre el tablero

3.3 Validación de impacto

`procesar_disparo()` compara la posición del pato contra la del disparo — en Python, comparar dos tuplas con `==` ya compara cada elemento automáticamente — y arma un registro con el resultado. Si coinciden se muestra la pantalla `ganador.jpeg` con el texto **WINNER**; si no, `gameover.jpeg` con **GAME OVER**.

3.4 El bucle principal

El juego siempre completa la cantidad de disparos configurada — **no se corta en el primer fallo** — porque las estadísticas finales necesitan la mayor cantidad de datos posible. Cada ronda dibuja un solo tablero con el pato y la mira juntos, y después muestra la pantalla de reacción a pantalla completa.

3.5 Resultados y CSV histórico

Al terminar la partida, la lista de registros se convierte en una tabla de pandas, se calcula el resumen (total de disparos, aciertos, fallos, porcentajes), y se muestra una pantalla final reutilizando `gameover.jpeg` con ese



Figura 3: Pantalla WINNER



Figura 4: Pantalla GAME OVER

resumen superpuesto. La partida se agrega a un archivo CSV histórico que acumula los resultados de todas las partidas jugadas, sin borrar las anteriores.



Figura 5: Pantalla final con el resumen de la partida

4. Funcionalidades implementadas

Repasando contra el enunciado original, punto por punto:

- ☑ Fondo del juego dibujado y mostrado en escala de grises.
- ☑ Cuadrícula generada dinámicamente (3×3, 4×4, 5×5, o cualquier $n \times n$ entre 3 y 8) usando estructuras matriciales.
- ☑ Función `pato()`: posición aleatoria con NumPy, dibujo del sprite.
- ☑ Función `pistola()`: posición de impacto aleatoria con NumPy, mira dibujada.
- ☑ Validación de impacto: imagen de acierto (WINNER) y de fallo (GAME OVER), con registro de cada resultado.
- ☑ Sistema de disparos configurable ($n = 20$ por defecto, ajustable desde el teclado al iniciar).
- ☑ Pantalla final con estadísticas: total, exitosos, errados, % de aciertos, % de fallos.
- ☑ Visualización estadística con tablas y gráficos (barras, pie chart, heatmap), usando pandas, Matplotlib y Seaborn.

5. Librerías utilizadas

El proyecto usa las cuatro librerías obligatorias (NumPy, pandas, Matplotlib, Seaborn) más Pillow como apoyo para cargar imágenes. La explicación detallada de **por qué se usó cada una, con ejemplos concretos del código**, está en un documento aparte: [LIBRERIAS.md](#).

Como resumen rápido:

Librería	Rol en el proyecto
NumPy	Matrices, cuadrículas, conversión a grises, aleatoriedad
pandas	Registro de disparos, resumen estadístico, CSV histórico
Matplotlib	Todo el dibujo: tablero, sprites, texto, pantallas

Librería	Rol en el proyecto
Seaborn	Gráficos estadísticos finales (barras, heatmap, leaderboard)
Pillow	Carga de imágenes desde disco

6. Análisis estadístico de resultados

Para este informe se jugó una partida de ejemplo con la configuración por defecto: grid 4×4 y 20 disparos. El resultado fue **1 acierto de 20 disparos (5 %)** — un resultado bajo, pero esperable: con 16 celdas posibles y una posición de disparo totalmente independiente de la del pato, la probabilidad de acertar cada disparo por puro azar es $1/16 = 6.25\%$, así que sobre 20 intentos lo más probable estadísticamente es acertar entre 1 y 2 veces. Esto es, en sí mismo, una observación interesante del proyecto: como ambas posiciones son aleatorias e independientes, el juego no mide puntería sino que ilustra un experimento de probabilidad — cada partida es, en el fondo, una simulación de Monte Carlo con muy pocas repeticiones.

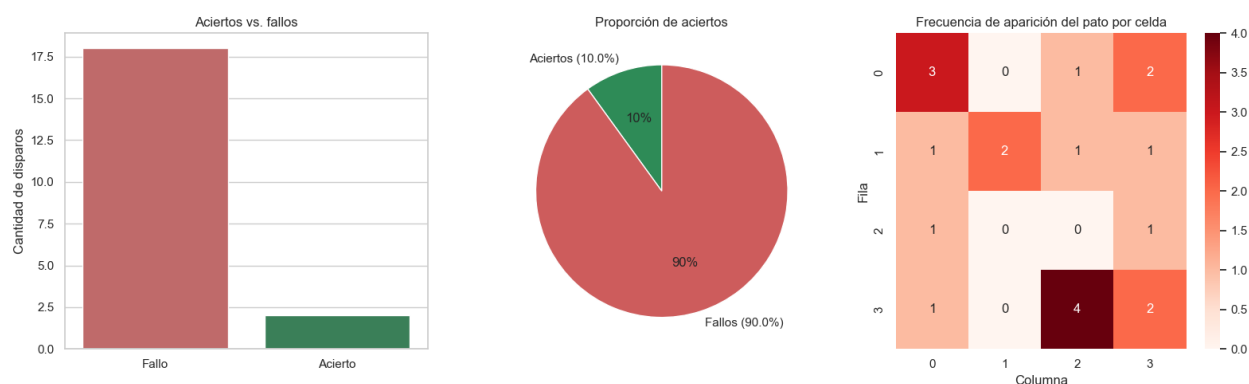


Figura 6: Gráfico de barras, pie chart y heatmap de la partida de ejemplo

El **gráfico de barras** y el **pie chart** muestran lo mismo desde dos ángulos: la proporción de aciertos frente a fallos. El **heatmap** es el más interesante desde el punto de vista de matrices: cuenta cuántas veces apareció el pato en cada celda del grid a lo largo de la partida — con más partidas jugadas, ese mapa de calor debería acercarse a una distribución uniforme, porque `pato()` elige la celda con `np.random.randint`, que da la misma probabilidad a cada celda.

Como la partida quedó guardada en el CSV histórico, la Sección 9 también arma un **leaderboard** comparando todas las partidas jugadas hasta el momento, ordenadas por porcentaje de aciertos — pensado para cuando distintos integrantes del equipo (o distintos compañeros del curso) prueben el notebook varias veces.

7. Decisiones de diseño

Algunas decisiones no son obvias mirando solo el código final, así que vale la pena dejarlas explícitas:

- **Solo escala de grises, no color ni blanco/negro simultáneos:** el enunciado pide mostrar el fondo en blanco/negro o escala de grises; el equipo optó por escala de grises únicamente, en todo el proyecto, para no complicar el código con versiones que no aportan a la lógica del juego.
- **pato() y pistola() simplificadas con imshow(extent=...):** una versión anterior mezclaba los píxeles del sprite a mano con una fórmula de composición alfa; se simplificó dejando que Matplotlib haga esa composición automáticamente, que es lo que la librería ya hace bien.
- **Sin splash de sangre ni sonido:** se consideraron en una etapa intermedia del desarrollo, pero se descartaron para mantener la lógica de validación de impacto simple y fácil de explicar — las pantallas WINNER/GAME OVER ya cumplen esa función.

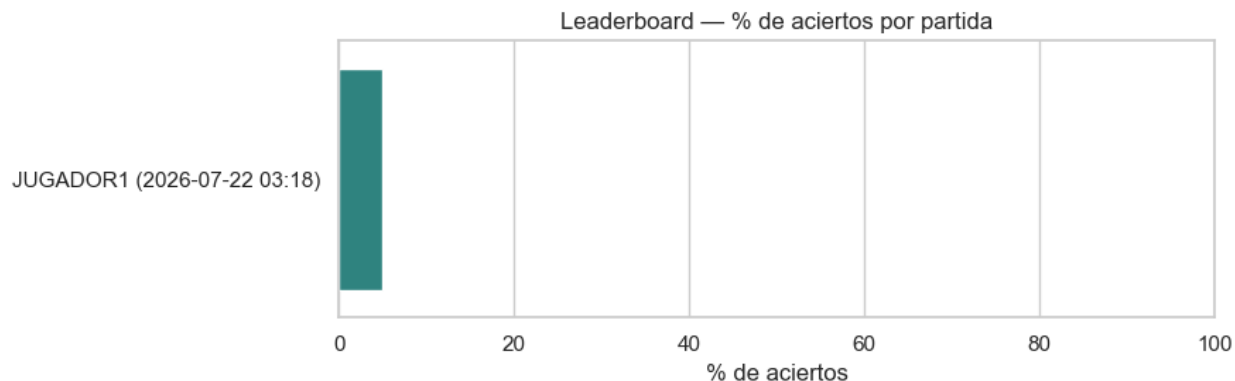


Figura 7: Leaderboard de partidas registradas

- **Configuración por teclado, no por una pantalla interactiva de botones:** se probó una pantalla de configuración estilo consola retro con botones (`ipywidgets`), pero depende de que el entorno donde corre el notebook tenga el renderizador de widgets bien configurado. `input()` es una función estándar de Python que funciona igual en cualquier notebook, así que fue la opción más confiable para algo tan central como la configuración inicial.

8. Conclusiones

El proyecto cumple con los cuatro pilares que pedía el enunciado: manipulación de matrices y arreglos con NumPy, gestión de datos estadísticos con pandas, creación de gráficos con Matplotlib y Seaborn, y una simulación computacional simple armada con funciones pequeñas y bien separadas por responsabilidad. Más allá de cumplir la consigna, el proceso de simplificar el código en varias iteraciones —sacando la composición alfa manual, el sonido, el splash de sangre— fue en sí mismo una lección práctica: la primera versión que funciona no siempre es la versión más fácil de explicar, y vale la pena revisar el código con esa pregunta en mente.

9. Cómo ejecutar el proyecto

```
pip install -r requirements.txt
jupyter notebook DuckHunt_Simulacion.ipynb
```

Correr las celdas en orden, de arriba hacia abajo; la Sección 2 va a pedir el nombre del jugador, el número de disparos y el tamaño del grid por teclado. Instrucciones más detalladas y capturas adicionales están documentadas en el [README.md](#) del repositorio.